PublicPulse: Real-Time Sentiment Intelligence Platform

Complete Architecture & Learning Roadmap

🎯 PROJECT GOAL (What Recruiters See)

You're building a **full-stack data engineering + ML platform** that demonstrates:

- ✅ **Data Engineering**: API ingestion → ETL → Data warehouse
- ✅ **Big Data**: Spark, Delta Lake, medallion architecture
- ✅ **NLP/ML**: Sentiment analysis, trend detection at scale
- ✅ **Real-Time**: Streaming, scheduling, orchestration
- ✅ **Full Stack**: Backend API, modern dashboard, databases
- ✅ **Production Thinking**: Error handling, monitoring, reproducibility
- ✅ **Lakehouse Architecture**: Bronze-Silver-Gold medallion pattern

This is **exactly** what Netflix, Uber, Airbnb's data teams do.

🏗️ COMPLETE SYSTEM ARCHITECTURE

```
┌──────────────────────────────────────────────────────────────────┐
│                         DATA SOURCES                             │
│  (Twitter/X, Reddit, NewsAPI - Real Data Sources)                │
└──────────────────────────┬───────────────────────────────────────┘
                           │
                           ↓
┌──────────────────────────────────────────────────────────────────┐
│              ORCHESTRATION LAYER (ANTIGRAVITY)                   │
│  - API ingestion workflows                                       │
│  - Scheduling (every 15 mins, hourly, daily)                     │
│  - Multi-step job triggers                                       │
│  - Error handling & retries                                      │
│  - Automated alerts                                              │
└──────────────────────────┬───────────────────────────────────────┘
                           │
                           ↓
┌──────────────────────────────────────────────────────────────────┐
│              PROCESSING LAYER (DATABRICKS)                       │
│                                                                  │
│  ┌──────────────────────────────────────────────────────────┐    │
```

```
│  │ BRONZE LAYER (Raw Data)                                  │   │
│  │ - Raw tweets, posts, articles stored as-is               │   │
│  │ - No transformations                                      │   │
│  │ - Preserves full history (immutable)                      │   │
│  │ Location: /mnt/delta/bronze/tweets, /bronze/reddit       │   │
│  └──────────────────────────────────────────────────────────┘   │
│              ↓                                                    │
│  ┌──────────────────────────────────────────────────────────┐   │
│  │ SILVER LAYER (Cleaned & Deduplicated)                    │   │
│  │ - Remove duplicates, nulls, invalid data                 │   │
│  │ - Extract text, timestamps, metadata                      │   │
│  │ - Normalize fields across sources                         │   │
│  │ - Create unified schema                                   │   │
│  │ Location: /mnt/delta/silver/posts_cleaned                │   │
│  └──────────────────────────────────────────────────────────┘   │
│              ↓                                                    │
│  ┌──────────────────────────────────────────────────────────┐   │
│  │ GOLD LAYER (Analytics Ready)                             │   │
│  │ - Sentiment scores (HuggingFace models)                  │   │
│  │ - Emotion classification                                  │   │
│  │ - Keyword extraction & frequency                          │   │
│  │ - Trend detection                                         │   │
│  │ - Aggregations (hourly, daily, weekly)                   │   │
│  │ Location: /mnt/delta/gold/sentiment_analytics            │   │
│  └──────────────────────────────────────────────────────────┘   │
│                                                                   │
│  Tools: Spark SQL, PySpark, MLlib, HuggingFace                   │
│  Storage: Delta Lake (ACID compliance, time travel)              │
│  Tracking: MLflow (model versions, predictions)                  │
└──────────────────────────┬──────────────────────────────────────┘
                           │
                           ↓

┌──────────────────────────────────────────────────────────────────┐
│              API LAYER (FastAPI Backend)                          │
│  - /api/sentiment/{topic} → real-time data                       │
│  - /api/trends/{topic} → trends over last N hours                │
│  - /api/keywords/{topic} → top keywords + frequency              │
│  - /api/forecast/{topic} → next 24h sentiment forecast           │
│  - WebSocket support for live updates                            │
└──────────────────────────┬──────────────────────────────────────┘
                           │
                           ↓

┌──────────────────────────────────────────────────────────────────┐
│         FRONTEND LAYER (Next.js Dashboard)                        │
│  - Real-time sentiment trends (line chart)                       │
│  - Source comparison (bar chart)                                 │
│  - Volume spikes timeline                                        │
```

```
│   - Word cloud of trending keywords                                  │
│   - Top viral posts                                                  │
│   - Sentiment forecast                                               │
│   - AI-generated insights                                            │
└──────────────────────────────────────────────────────────────────────┘
```

🔍 WHY EACH TOOL (The Learning Part)

ANTIGRAVITY: Why Orchestration Matters

The Problem Without Orchestration:

```python
# ❌ BAD: Hardcoded script approach
if __name__ == "__main__":
    tweets = fetch_twitter()
    reddit = fetch_reddit()
    sentiment = classify_sentiment(tweets + reddit)
    save_to_databricks(sentiment)
    send_alerts(sentiment)
    generate_report(sentiment)
```

Issues:

- If step 4 fails, you lose steps 1-3
- No retry logic = manual reruns
- No visibility into what succeeded/failed
- Can't schedule different steps differently
- No dependency tracking
- Nightmare for debugging in production

What Antigravity Solves:

- **Workflow Orchestration**: Define dependencies (step B runs only after A succeeds)
- **Scheduling**: Cron-like scheduling, complex schedules
- **Retry Logic**: Auto-retry failed steps (essential for flaky APIs)
- **Monitoring**: See which job ran when, what failed, why
- **Triggering**: When new data arrives in one system, trigger downstream jobs
- **Multi-Cloud**: Coordinate jobs across Databricks, AWS, GCP, etc.

Real Companies Use Airflow/Dagster/Antigravity Because:

- Netflix has 10,000+ DAGs (directed acyclic graphs)
- Uber runs 50,000 jobs daily
- Without orchestration, one API failure cascades

When to Use:

- ✅ Multiple data sources → must coordinate ingestion
- ✅ Steps depend on each other → need guaranteed order
- ✅ Jobs fail sometimes → need retries + alerts
- ✅ Need scheduled runs → cron jobs insufficient

DATABRICKS: Why Spark + Delta Lake

The Problem Without Databricks (Using Pandas Locally):

```python
import pandas as pd
df = pd.read_csv("tweets.csv")  # 500GB file = CRASH
sentiment = classify_sentiment(df)  # Hours on 1 machine
```

Issues:

- Can't load 500GB into RAM (pandas is single-machine)
- 10 hours to process what Spark does in 10 minutes
- No fault tolerance (machine crashes = start over)
- Can't scale—next month data doubles, script breaks

What Databricks Solves:

1. **Distributed Computing (Spark)**
    - Split data across 100 machines
    - Process 500GB in 10 minutes instead of 10 hours
    - Auto-fail-over if a machine dies
2. **Delta Lake (Data Lakehouse)**
    - ACID transactions (no data corruption)
    - Time travel (query data from yesterday)
    - Schema evolution (add columns safely)
    - Unified source of truth for Analytics + ML
3. **Medallion Architecture (Bronze-Silver-Gold)**

- **Bronze**: Raw, messy data (100% historical record)
- **Silver**: Cleaned, normalized data (single source of truth)
- **Gold**: Pre-aggregated analytics tables (fast dashboards)

Real companies use this because:

- Easy rollback: "Bugs in gold? Regenerate from silver"
- Audit trail: Can prove where every number came from
- Reusability: Multiple teams use same silver/gold tables

4. **MLflow Integration**
   - Track model versions automatically
   - Reproduce which model made which prediction
   - A/B test different sentiment classifiers

When Databricks is Overkill vs Ideal:

Scenario	Size	Databricks?	Alternative
100 tweets/day, local analysis	<1GB	❌ Overkill	Local Python
10K tweets/day, SQL analytics	10GB	⚠️ Maybe	PostgreSQL
1M tweets/day, real-time trends	100GB+	✅ Essential	None (Databricks required)
100M events/day, ML pipeline	1TB+	✅ MUST	Nothing else scales

FASTAPI Backend: Why REST API

For the Dashboard to Work:

- Frontend needs `/api/sentiment/{topic}` endpoint
- Real-time → WebSocket for live updates
- Dashboard shouldn't query Databricks directly (security + performance)
- API layer = caching + rate limiting

What to Expose:

```
GET /api/sentiment/tesla?hours=24
→ Last 24h sentiment data

GET /api/trends/bitcoin?period=7d
→ Weekly trend, momentum, direction

GET /api/keywords/openai
→ Top 50 keywords, frequency, sentiment

WebSocket /ws/live/apple
→ Real-time sentiment as it changes
```

Next.js Dashboard: Why Not Just Jupyter?

Jupyter is for **exploration**. Dashboard is for **communication**.

Recruiters notice:

- ✅ Polished UI (not default Matplotlib)
- ✅ Real-time interactivity
- ✅ Professional design
- ✅ Mobile responsive

📊 WHEN TO USE PYTHON vs SQL vs PYSPARK

Task	Tool	Why
Fetch from Reddit API	Python	API clients are Python libraries
Clean 500K records	PySpark SQL	Distributed, 100x faster than pandas
Calculate daily aggregations	SQL	Native SQL, faster + simpler
Train sentiment model	Python locally first	Iterate quickly on laptop
Inference on 10M records	PySpark + HuggingFace	Distributed, parallelizable
Exploratory analysis	Pandas locally	Fast iteration on small subsets
Production ETL	Spark notebook	Scalable, version-controlled

🚀 PHASE-BASED ROADMAP

PHASE 1: MVP (1 Week - Minimum Viable Product)

Goal: Build end-to-end with minimal data, prove the concept works.

What to Build:

1. Fetch 100 tweets on 1 topic (no real API calls yet, mock data)
2. Basic sentiment classification (using pretrained model, 5 lines of code)
3. Simple dashboard with 3 charts
4. Manual Databricks notebook (no Antigravity yet)

Why This Phase Matters:

- Get full stack working before adding complexity
- Prove you can orchestrate data → model → dashboard
- Portfolio foundation: "I built end-to-end"

Skills Gained:

- Databricks notebooks basics
- PySpark SQL syntax
- FastAPI GET endpoints
- Next.js charts

Resume Impact: ⭐⭐ (Shows you can build something, but not production-grade yet)

Time: 2-3 days

PHASE 2: Strong Portfolio Version (1-2 Weeks)

Goal: Production-ready, recruiters impressed, real data sources.

What to Add:

1. **Antigravity Orchestration**
    - Schedule tweet fetch every 15 mins
    - Auto-retry on failure
    - Monitor pipeline status
2. **Multi-Source Integration**
    - Twitter/X API (real keys, rate limiting)
    - Reddit API (real data)

- NewsAPI (optional, real articles)

3. **Advanced NLP**
    - Sentiment + Emotion classification
    - Keyword extraction (TF-IDF)
    - Trend detection (momentum scores)
4. **Better Dashboard**
    - Real-time updates (WebSocket)
    - Sentiment over time (beautiful chart)
    - Word cloud from keywords
    - Top viral posts
    - Source comparison
5. **Backend Improvements**
    - Caching (Redis)
    - Rate limiting per user
    - Error handling
    - Logging

Skills Gained:

- Antigravity DAG design
- Real API integration + rate limiting
- Advanced Spark SQL
- Real-time data processing
- WebSocket backend
- Professional dashboard design

Resume Impact: ⭐⭐⭐⭐⭐ (This is what gets callbacks)

Time: 1-2 weeks

PHASE 3: Enterprise-Level Version (3+ Weeks)

Goal: Something you'd actually deploy to production.

What to Add:

1. **Advanced Orchestration**
    - A/B test different sentiment models
    - Trigger alerts when sentiment drops below threshold

- Generate daily reports automatically
- Multi-step workflows: ingest → clean → model → forecast → alert

2. **ML Pipeline**
    - Fine-tune sentiment model on domain-specific data
    - MLflow experiment tracking
    - Model registry + versioning
    - Automated retraining on new data
3. **Advanced Analytics**
    - Sentiment forecasting (Prophet/LSTM)
    - Anomaly detection (sudden sentiment swings)
    - Causal analysis (what events caused sentiment shifts)
    - Sentiment by geographic region (if data available)
4. **Enterprise Features**
    - User authentication
    - Team workspaces
    - Custom alerts per user
    - Saved searches + subscriptions
    - Data export (CSV, PDF reports)
    - API rate limiting + key management
5. **DevOps**
    - Dockerization
    - CI/CD pipeline (GitHub Actions)
    - Environment configs (dev/staging/prod)
    - Monitoring + alerts (PagerDuty)
    - Database backups
6. **Advanced Dashboard**
    - Multi-topic comparison
    - Custom date ranges
    - Sentiment by source breakdown
    - Prediction confidence intervals
    - Interactive filters
    - Export reports

Skills Gained:

- ML ops + model deployment

- Advanced Spark (window functions, streaming)
- Advanced orchestration (conditional workflows, dynamic parallelism)
- DevOps basics
- System design

Resume Impact: ⭐⭐⭐⭐⭐⭐ (This is a job offer level project)

Time: 3-4 weeks

📁 FOLDER STRUCTURE (Phase 2)

```
publicpulse/
├── README.md                          # Start here
├── ARCHITECTURE.md                    # This file
├── requirements.txt                   # Python deps
├── docker-compose.yml                 # Local dev setup
│
├── backend/
│   ├── main.py                        # FastAPI app
│   ├── config.py                      # Config (API keys, etc)
│   ├── routers/
│   │   ├── sentiment.py               # /api/sentiment
│   │   ├── trends.py                  # /api/trends
│   │   └── keywords.py                # /api/keywords
│   ├── services/
│   │   ├── databricks_connector.py    # Query gold tables
│   │   ├── sentiment_classifier.py    # HuggingFace inference
│   │   └── cache.py                   # Redis caching
│   └── websocket/
│       └── live_sentiment.py          # Real-time WS
│
├── databricks/
│   ├── notebooks/
│   │   ├── 01_bronze_ingestion.py    # Fetch raw data
│   │   ├── 02_silver_cleaning.py     # Clean + dedupe
│   │   ├── 03_gold_sentiment.py      # Classify sentiment
│   │   └── 04_gold_analytics.py      # Aggregations
│   └── config/
│       └── spark_config.py            # Spark settings
│
├── orchestration/
│   ├── antigravity/
│   │   ├── dag_ingest_sentiment.yaml  # Main DAG
│   │   ├── dag_forecast.yaml          # Forecast job
```

```
│   │   └── slack_alerts.yaml          # Alert workflow
│   └── scripts/
│       ├── create_dags.py             # Deploy DAGs
│       └── monitor.py                 # Check status
│
├── frontend/
│   ├── pages/
│   │   ├── index.tsx                  # Main dashboard
│   │   └── api/
│   │       └── [...proxy].ts          # API proxy
│   ├── components/
│   │   ├── SentimentChart.tsx         # Line chart
│   │   ├── WordCloud.tsx              # Word cloud
│   │   ├── TopPosts.tsx               # Viral posts list
│   │   └── SourceComparison.tsx       # Bar chart
│   ├── styles/
│   │   └── globals.css                # Theme + design
│   └── lib/
│       └── api.ts                     # API client
│
├── tests/
│   ├── test_backend.py                # Unit tests
│   ├── test_spark.py                  # Spark job tests
│   └── test_e2e.py                    # End-to-end
│
├── data/
│   ├── sample_tweets.json             # Mock data for testing
│   └── sample_reddit.json
│
└── docs/
    ├── INTERVIEW_GUIDE.md             # How to explain this
    ├── DEPLOYMENT.md                  # How to deploy
    └── RESUME_BULLETS.md              # What to tell recruiters
```

⏳ BUILD ORDER (Phase 2: Do This Sequence)

This order minimizes rework:

Week 1: Foundation

Day 1-2: Mock Data Setup + Databricks Bronze

- Create mock tweet/reddit JSON files
- Upload to Databricks DBFS
- Write `01_bronze_ingestion.py` notebook (just copies raw data to Delta)

- Goal: Raw data in Bronze table ✓

Day 3: Silver Layer

- Write `02_silver_cleaning.py` (remove nulls, normalize schema)
- Test on mock data
- Goal: Cleaned table in Silver ✓

Day 4: Sentiment Inference

- Write `03_gold_sentiment.py` (use HuggingFace, add sentiment column)
- Test sentiment on sample posts
- Goal: Sentiment scores in Gold ✓

Day 5: FastAPI Backend Setup

- Create `main.py` with 1 endpoint `/api/sentiment/{topic}`
- Connect to Databricks (simple query)
- Goal: API returns data from Gold table ✓

Why This Order:

- Get data pipeline working before frontend
- Frontend has real API to query
- Each step is testable independently

Week 2: Real Integration + Polish

Day 6-7: Antigravity Orchestration

- Create `dag_ingest_sentiment.yaml` that runs all 4 notebooks in order
- Schedule every 15 minutes
- Add error handling + Slack alerts
- Goal: Automated pipeline ✓

Day 8: Multi-Source Integration

- Add Twitter/Reddit actual API calls
- Handle rate limiting
- Update `01_bronze_ingestion.py` to fetch real data
- Goal: Real data flowing through pipeline ✓

Day 9: Dashboard MVP

- Next.js page with 3 charts:
  - Sentiment line chart
  - Top keywords word cloud
  - Top posts table
- Goal: Vizualize data ✓

Day 10: Polish

- WebSocket for real-time updates
- Add caching (Redis)
- Professional styling
- Goal: Production-ready ✓

🎓 WHAT RECRUITERS CARE ABOUT MOST

Most Impressive (Talk About First)

1. **You orchestrated data pipelines** (Antigravity DAGs)
   - "I built a workflow that fetches from 3 APIs, cleans data, runs ML inference, and alerts on anomalies—all automated."
2. **You used a real data warehouse** (Databricks + Delta Lake)
   - "I stored 1M+ records in Delta Lake using medallion architecture, so data is ACID-compliant and auditable."
3. **You built an actual dashboard** (not Jupyter notebooks)
   - "I built a real-time dashboard with WebSockets, not just plots."
4. **You handled scale** (Spark, not pandas)
   - "I could process 10B records if needed because I used PySpark, not pandas."

Intermediate (Mention If Asked)

- HuggingFace NLP models
- Redis caching
- Error handling + retries
- Databricks MLflow tracking

- "I made a sentiment analysis model" (everyone does this)
- "I made charts" (so does Excel)
- "I used an API" (basic)

⚠️ COMMON BEGINNER MISTAKES (Avoid These!)

Mistake 1: Using Pandas on 100GB Data

```python
# ❌ WRONG
df = pd.read_csv("100gb_tweets.csv")
df['sentiment'] = df['text'].apply(classify_sentiment)
```

→ This will CRASH. Use Spark instead.

Fix: Use PySpark

```python
# ✅ RIGHT
df = spark.read.csv("tweets.csv", header=True)
df = df.withColumn("sentiment", classify_sentiment_udf(col("text")))
```

Mistake 2: Hardcoded API Keys

```python
# ❌ WRONG
API_KEY = "sk_live_4eC39HqLyjWDarhtT881g95K"  # Visible in GitHub!
```

Fix: Use environment variables + secrets manager

```python
# ✅ RIGHT
API_KEY = os.getenv("TWITTER_API_KEY")
# Store in .env (local) or Databricks Secrets (production)
```

Mistake 3: No Error Handling

```python
# ❌ WRONG
tweets = fetch_twitter_api()  # What if API is down?
```

Fix: Retry logic + fallback

```python
# ✅ RIGHT
@retry(max_attempts=3, backoff=2)
def fetch_twitter_api():
    try:
        return api.search(...)
    except RateLimitError:
        time.sleep(60)  # Wait, retry
        return api.search(...)
```

Mistake 4: Assuming Data Quality

```python
# ❌ WRONG
sentiment_avg = df['sentiment'].mean()  # NULL values = wrong answer
```

Fix: Validate + clean

```python
# ✅ RIGHT
df = df.filter(col('text').isNotNull() & (col('text') != ''))
sentiment_avg = df['sentiment'].mean()
```

Mistake 5: No Tests

```python
# ❌ WRONG
# Push code to production, hope it works
```

Fix: Basic unit tests

```python
```

```python
# ✅ RIGHT
def test_sentiment_classifier():
    assert classify_sentiment("I love this!") > 0.7  # Positive
    assert classify_sentiment("I hate this") < 0.3    # Negative
```

Mistake 6: Monolithic Notebooks

```python
# ❌ WRONG
# One 500-line Databricks notebook doing everything
```

Fix: Modular notebooks

```python
# ✅ RIGHT
# 01_ingest.py - just fetch data
# 02_clean.py - just clean
# 03_sentiment.py - just inference
# 04_aggregate.py - just aggregations
```

Why: Easy to debug, test independently, reuse.

Mistake 7: Rebuilding Instead of Materializing

```python
# ❌ WRONG (Slow)
def get_sentiment_trend():
    # Query raw data every time
    raw = spark.read.parquet("bronze/tweets")
    cleaned = raw.filter(...)
    sentiment = cleaned.withColumn("sentiment", ...)
    return sentiment.groupBy("date").agg(...)
```

Fix: Pre-compute once (Gold layer)

```python
# ✅ RIGHT (Fast)
gold = spark.read.delta("gold/sentiment_daily")  # Already computed
return gold.filter(...)
```

🏆 PRODUCTION BEST PRACTICES

1. Idempotency: Run Same Job 10 Times = Same Result

```python
# ❌ WRONG
df.write.mode("append").save("gold/sentiment")  # Duplicates if run twice
```

```python
# ✅ RIGHT
df.write.mode("overwrite").option("replaceWhere", "date='2024-04-18'").saveAsTable("
# Same result whether run once or 10 times
```

---

2. Data Lineage: Track Where Data Comes From

```python
# Add metadata
df = df.withColumn("_source_file", input_file_name())
df = df.withColumn("_loaded_at", current_timestamp())
df = df.withColumn("_notebook_version", lit("v1.2"))
```

---

3. Logging, Not Print Statements

```python
# ❌ WRONG
print("Processing tweets...")

# ✅ RIGHT
import logging
logger = logging.getLogger(__name__)
logger.info("Processing tweets...")  # Searchable, timestamped
```

---

4. Alerts Before Crashes

```python
```

```
# ✅ RIGHT
if df.count() < 100:
    send_slack_alert("WARNING: Only 100 tweets today, check API")
    return None
```

---

5. Data Quality Checks (Great Catch Metric)

```python
# In gold layer
# Check: No NULL sentiments
null_count = df.filter(col('sentiment').isNull()).count()
assert null_count == 0, f"Found {null_count} NULL sentiments!"

# Check: Sentiment is between 0-1
assert df.agg(min('sentiment'), max('sentiment')).collect()[0][0] >= 0
```

💬 HOW TO EXPLAIN IN INTERVIEWS

Interviewer: "Walk us through your sentiment analysis project."

Your Answer (2-3 minutes):

"I built PublicPulse, an end-to-end data platform that analyzes sentiment across social media.

The Architecture:

- I use **Antigravity** to orchestrate pipelines: every 15 minutes, it fetches tweets and Reddit posts, kicks off Databricks jobs, monitors for failures, and alerts on anomalies.
- In **Databricks**, I implemented a medallion architecture: Bronze stores raw data, Silver cleans it, Gold adds sentiment scores using HuggingFace transformers—all as Delta tables for ACID compliance.
- I built a **FastAPI backend** that queries the Gold layer and exposes `/api/sentiment` endpoints for my Next.js dashboard.
- The **dashboard** shows real-time sentiment trends, top keywords, viral posts, and forecasts.

Why These Choices:

- Antigravity because coordinating 3 data sources manually is error-prone; I needed retries and visibility.
- Databricks because 1M+ records don't fit in pandas; Spark parallelizes across 10 machines.
- Delta Lake because ACID guarantees mean no data corruption—important for analytics.

- Medallion architecture because it's audit-friendly: I can trace any wrong number back to raw data.

What I Learned:

- The difference between data engineering (pipelines) and data science (models)
- Why orchestration tools matter at scale
- How to design for reproducibility

If in production, I'd add:

- Model A/B testing (which sentiment classifier is more accurate?)
- Anomaly detection (alert when sentiment suddenly drops)
- Geographic breakdown
- User authentication"

Interviewer: "Why Databricks over Snowflake?"

"Both are data warehouses, but Databricks is better for **compute-heavy workloads**:

- I need to run HuggingFace models on 1M records—that's CPU-intensive
- Spark's built-in ML libraries integrate seamlessly
- Delta Lake + Spark are tightly integrated

Snowflake would be better if I only did SQL analytics without ML."

Interviewer: "How would you scale this to 100M tweets/day?"

"Great question. Currently:

- Databricks auto-scales workers up/down
- Spark partitions data by date, so queries scan only needed data

If 100x growth:

- Partition by date AND source (Redis for hot tweets)
- Stream ingestion instead of batch (Kafka → Spark Streaming)
- Use MLflow to version models, reuse predictions (avoid recomputing)
- Add data archival (30-day hot, older to S3)"

📝 RESUME BULLETS (Copy-Paste Worthy)

Phase 2 Version:

- **Built end-to-end sentiment analysis platform** (Antigravity → Databricks → FastAPI → Next.js dashboard) processing real tweets & Reddit data with 99.9% uptime
- **Implemented medallion architecture** (Bronze-Silver-Gold) on Delta Lake for 1M+ records, ensuring ACID compliance and audit trails
- **Orchestrated data pipelines** using Antigravity DAGs with auto-retry logic, Slack alerts, and 15-minute refresh cadence
- **Deployed real-time dashboard** with WebSocket support, sentiment trends, keyword analysis, and viral post detection
- **Integrated HuggingFace transformers** for distributed sentiment/emotion classification across 1M+ social media posts using PySpark

Phase 3 Version (After Adding ML Ops):

- **Designed ML ops pipeline** with MLflow experiment tracking, model registry, and automated retraining—achieving 4% accuracy improvement through A/B testing
- **Built production orchestration system** orchestrating 10+ workflows (data ingestion, model inference, forecasting, alerting) with 99.95% SLA
- **Implemented advanced analytics** including anomaly detection, sentiment forecasting (Prophet), and causal analysis
- **Architected multi-tenant system** with user authentication, team workspaces, and per-user customizable alerts

---

🔗 WHAT'S NEXT

This architecture document is your **blueprint**. Now we'll build:

1. **Phase 1**: Mock data + basic pipeline (tomorrow)
2. **Phase 2**: Real integration + dashboard (next 1 week)
3. **Phase 3**: Advanced features + deployment (optional)

Ready to start coding?

📚 KEY TERMS (For Interviews)

- **Medallion Architecture**: Bronze (raw) → Silver (cleaned) → Gold (analytics)

- **Delta Lake**: ACID-compliant data format, version control for data
- **Spark**: Distributed computing framework (100x faster than pandas on large data)
- **DAG**: Directed Acyclic Graph—task dependency structure
- **Idempotency**: Running 10 times = same result as running once
- **HuggingFace**: Pre-trained NLP models (sentiment, classification, etc.)
- **ACID**: Atomicity, Consistency, Isolation, Durability (data reliability)
- **Lakehouse**: Hybrid of data lake (cheap, flexible) + warehouse (fast, structured)

Next Steps:

1. Review this architecture
2. Create folder structure locally
3. Start Phase 1 (mock data setup)
4. I'll provide full code for each component

Let me know when ready!